

Proof of Concept: HTTP Log Attack Analysis

Splunk-based Investigation of http logs

Date: September 17, 2026

1. Objective

This POC documents the investigation of the HTTP connection log (sourcetype="http logs") imported into Splunk, to identify, validate, and quantify web-based attack activity on the internal network. Two distinct attacks were identified. Each finding below is backed by the exact SPL query used, a screenshot of the result returned by Splunk, and the reasoning used to conclude whether the result confirms an attack.

2. Data Source

- HTTP log — sourcetype="http logs", Zeek HTTP connection records, imported into Splunk as raw unstructured text.

Because the file was ingested without field extraction configured, every query below begins by extracting the needed fields from _raw using rex before any analysis is performed.

3. Attack #1 — DirBuster Directory Brute-Force (192.168.203.63)

Investigation approach: I started the investigation by looking at the overall HTTP traffic and identifying source IPs that generated unusually large volumes of requests. I did not assume the attack type at this stage. The purpose of this first step was to identify an abnormal source that could then be investigated further.

3.1 Main Evidence — Top Source by Volume

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t+])\t(?<uid>[^\t+])\t(?<src_ip>[^\t+])\t(?<src_port>[^\t+])\t(?<dest_ip>[^\t+])\t(?<dest_port>[^\t+])\t(?<trans_depth>[^\t+])\t(?<method>[^\t+])\t(?<host>[^\t+])\t(?<uri>[^\t+])\t(?<referrer>[^\t+])\t(?<user_agent>[^\t+])\t(?<req_len>[^\t+])\t(?<resp_len>[^\t+])\t(?<status_code>[^\t+])\t(?<status_msg>[^\t+])\t(?<info_code>[^\t+])\t(?<info_msg>[^\t+])\t(?<filename>[^\t+])\t(?<tags>[^\t+])\t(?<username>[^\t+])\t(?<password>[^\t+])"
| stats count as events, dc(uri) as unique_uris by src_ip
| sort -events
```

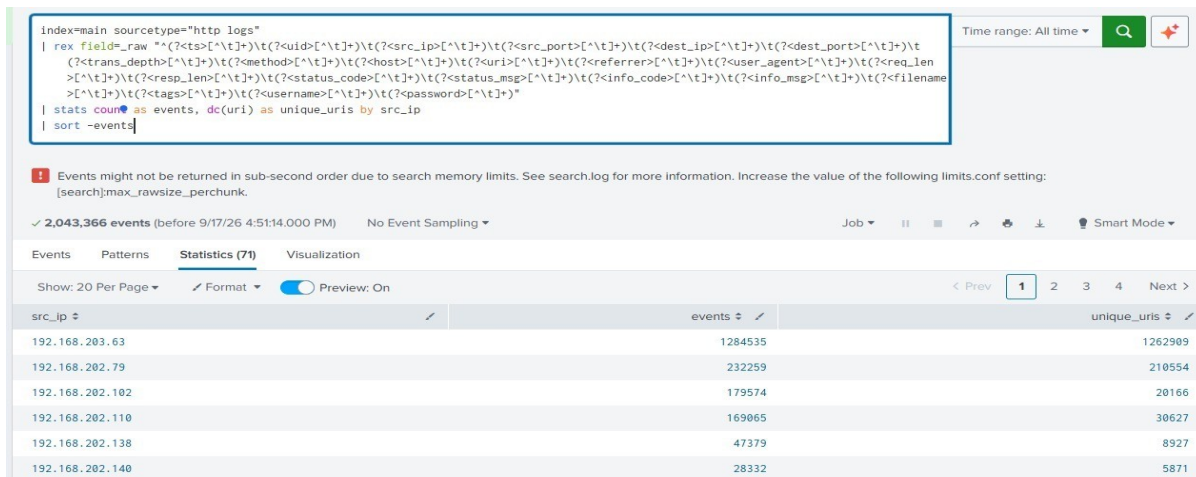


Figure 1 — 192.168.203.63 tops the list with 1,284,535 events and 1,262,909 unique URIs — by far the largest source in the entire log, out of 2,043,366 total events.

A single source generating 1.28 million requests — over 62% of the entire log — with over a million distinct URI guesses, is not normal browsing. This is the core evidence for this attack; the sections below present the supporting evidence gathered to validate it.

3.2 Supporting Evidence

Investigation approach: After identifying 192.168.203.63 as the highest-volume source, I wanted to understand what was generating the requests. I therefore examined the User-Agent field to determine whether the traffic was coming from a normal browser or an automated tool.

3.2.1 Tool Signature Confirmation

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t(?<trans_depth>[^\t]+\t(?<method>[^\t]+\t(?<host>[^\t]+\t(?<uri>[^\t]+\t(?<referrer>[^\t]+\t(?<user_agent>[^\t]+\t(?<req_len>[^\t]+\t(?<resp_len>[^\t]+\t(?<status_code>[^\t]+\t(?<status_msg>[^\t]+\t(?<info_code>[^\t]+\t(?<info_msg>[^\t]+\t(?<filename>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+)"
| search src_ip="192.168.203.63"
| stats count by user_agent
| sort -count
```

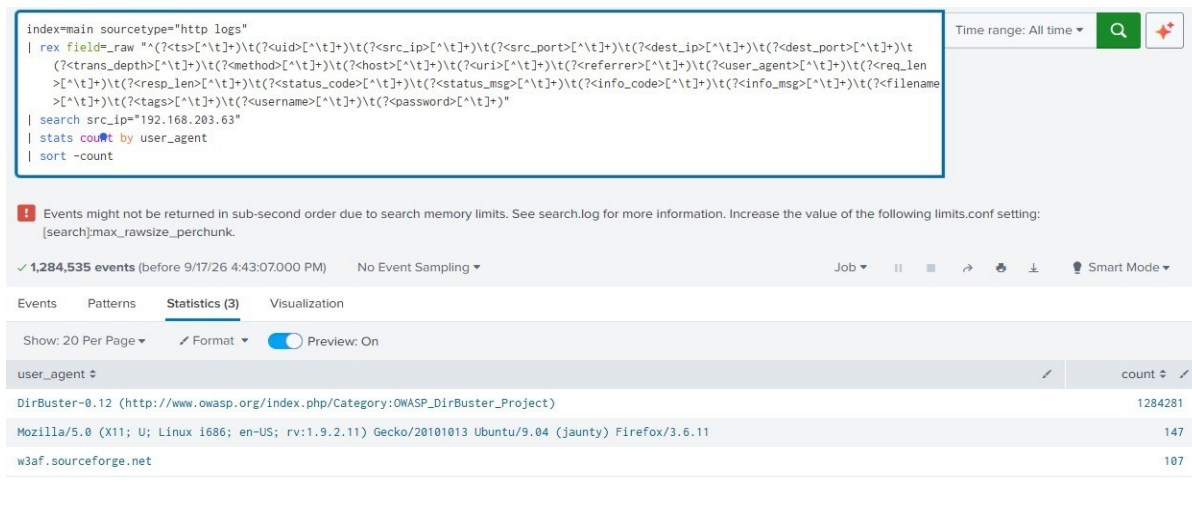


Figure 2 — Of 1,284,535 requests from this source, 1,284,281 carry the User-Agent “DirBuster-0.12 (http://www.owasp.org/index.php/Category:OWASP_DirBuster_Project)” — a real, well-known directory-brute-forcing tool.

Investigation approach: After seeing the DirBuster signature, I next checked the destination IPs to understand whether the source was scanning multiple systems or concentrating its activity on a particular web server.

3.2.2 Target Concentration

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t]+\t)(?<uid>[^\t]+\t)(?<src_ip>[^\t]+\t)(?<src_port>[^\t]+\t)(?<dest_ip>[^\t]+\t)(?<dest_port>[^\t]+\t)\t(?<trans_depth>[^\t]+\t)(?<method>[^\t]+\t)(?<host>[^\t]+\t)(?<uri>[^\t]+\t)(?<referrer>[^\t]+\t)(?<user_agent>[^\t]+\t)(?<req_len>[^\t]+\t)(?<resp_len>[^\t]+\t)(?<status_code>[^\t]+\t)(?<status_msg>[^\t]+\t)(?<info_code>[^\t]+\t)(?<info_msg>[^\t]+\t)(?<filename>[^\t]+\t)(?<tags>[^\t]+\t)(?<username>[^\t]+\t)(?<password>[^\t]+)"
| search src_ip="192.168.203.63"
| stats count by dest_ip
| sort -count
```

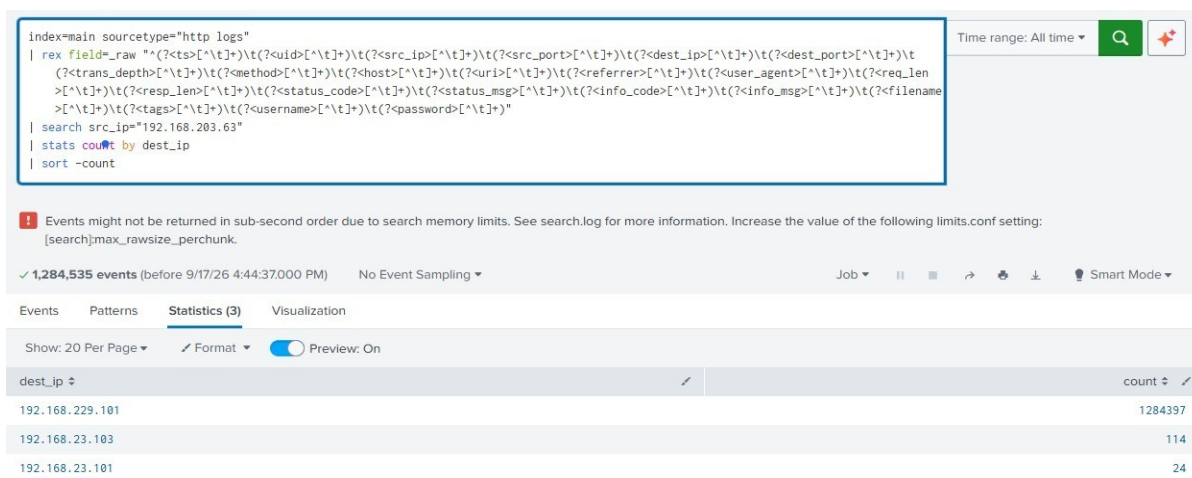


Figure 3 — 1,284,397 of this source’s 1,284,535 requests — nearly all of them — were aimed at a single host, 192.168.229.101. Only 114 and 24 requests touched two other hosts.

Investigation approach: Since the activity looked like directory enumeration, I then examined the HTTP response status codes. This helped determine whether the requests were finding existing resources or repeatedly guessing paths that did not exist.

3.2.3 Blind-Enumeration Confirmation (Status Codes)

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t+])\t(?<uid>[^\t+])\t(?<src_ip>[^\t+])\t(?<src_port>[^\t+])\t(?<dest_ip>[^\t+])\t(?<dest_port>[^\t+])\t(?<trans_depth>[^\t+])\t(?<method>[^\t+])\t(?<host>[^\t+])\t(?<uri>[^\t+])\t(?<referrer>[^\t+])\t(?<user_agent>[^\t+])\t(?<req_len>[^\t+])\t(?<resp_len>[^\t+])\t(?<status_code>[^\t+])\t(?<status_msg>[^\t+])\t(?<info_code>[^\t+])\t(?<info_msg>[^\t+])\t(?<filename>[^\t+])\t(?<tags>[^\t+])\t(?<username>[^\t+])\t(?<password>[^\t+])"
| search src_ip="192.168.203.63"
| stats count by status_code
| sort -count
```

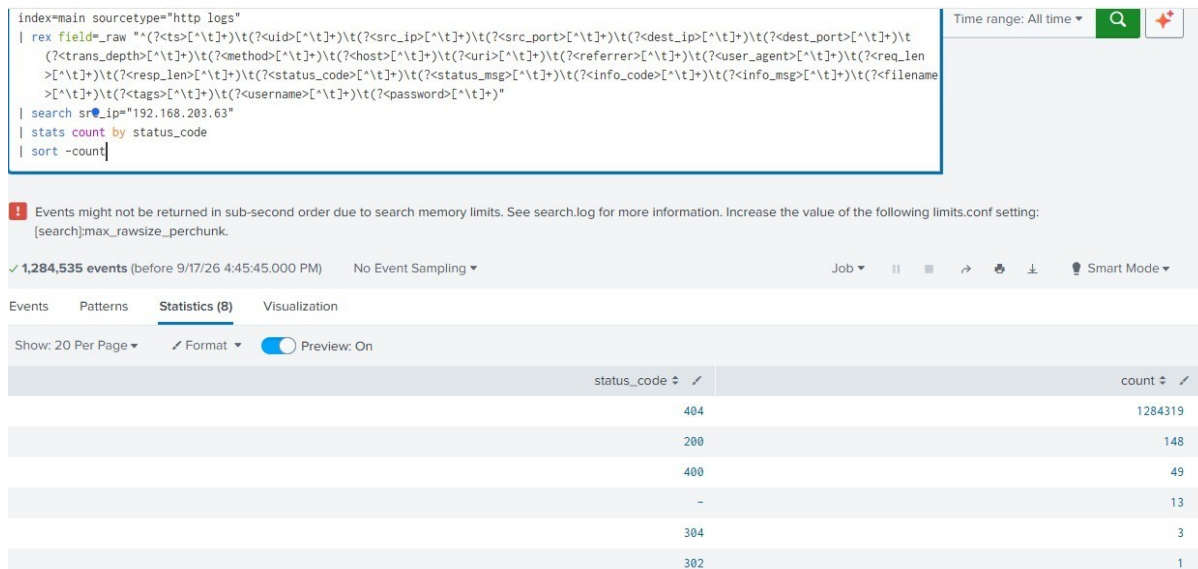


Figure 4 — 1,284,319 of 1,284,535 requests (99.98%) returned 404 Not Found. Only a handful returned 200, 400, or other codes — confirming this was blind guessing, not real browsing of existing content.

Together, the tool signature, the near-total concentration on a single host, and the near-total 404 failure rate confirm this was an automated directory brute-force scan carried out with DirBuster — not legitimate traffic.

4. Attack #2 — Web Enumeration & Multi-Vector Exploitation (192.168.202.79)

Investigation approach: After validating the first attack, I continued examining the dataset for other sources showing abnormal HTTP behavior. I found 192.168.202.79 generating a large number of requests, so I measured its request volume, number of destination hosts, and number of unique URIs to determine whether the activity represented systematic enumeration.

4.1 Main Evidence 1 — Enumeration Volume

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t(?<trans_depth>[^\t]+\t(?<method>[^\t]+\t(?<host>[^\t]+\t(?<uri>[^\t]+\t(?<referrer>[^\t]+\t(?<user_agent>[^\t]+\t(?<req_len>[^\t]+\t(?<resp_len>[^\t]+\t(?<status_code>[^\t]+\t(?<status_msg>[^\t]+\t(?<info_code>[^\t]+\t(?<info_msg>[^\t]+\t(?<filename>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+))"
| search src_ip="192.168.202.79"
| stats count as events, dc(dest_ip) as targets, dc(uri) as unique_uris
```

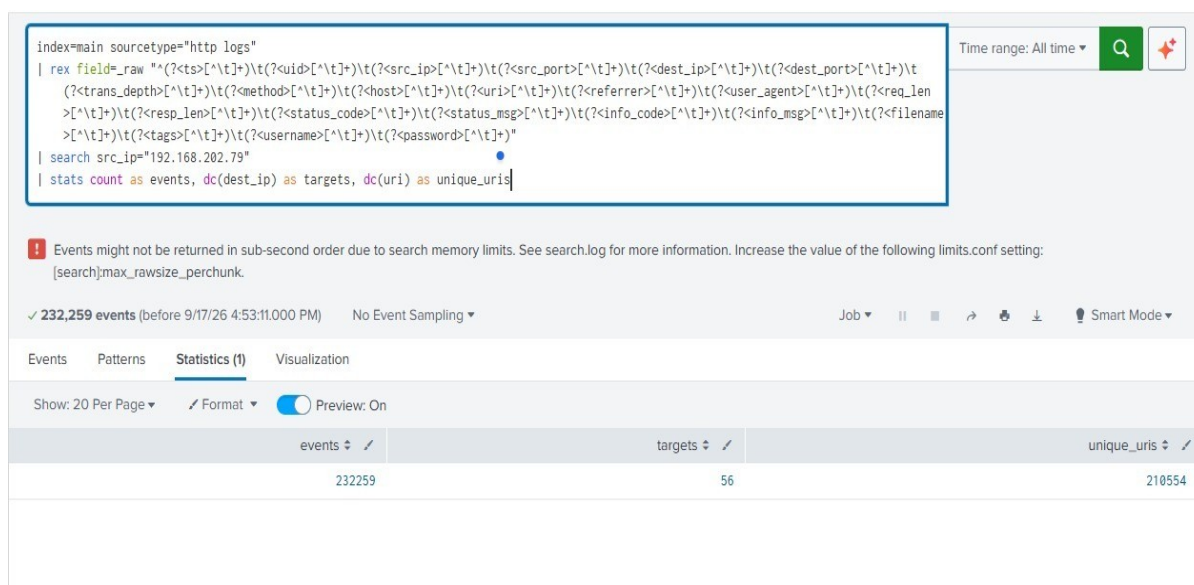


Figure 5 — 192.168.202.79 made 232,259 requests against 56 different destination hosts, using 210,554 unique URIs — a systematic enumeration sweep, not normal browsing.

Investigation approach: The large number of unique URIs and destination hosts indicated systematic enumeration, so I examined the URI and referrer values for suspicious patterns. The purpose was to determine whether the activity had moved beyond reconnaissance into attempts to exploit web applications.

4.2 Main Evidence 2 — Injection / Traversal Payloads

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t(?<trans_depth>[^\t]+\t(?<method>[^\t]+\t(?<host>[^\t]+\t(?<uri>[^\t]+\t(?<referrer>[^\t]+\t(?<user_agent>[^\t]+\t(?<req_len>[^\t]+\t(?<resp_len>[^\t]+\t(?<status_code>[^\t]+\t(?<status_msg>[^\t]+\t(?<info_code>[^\t]+\t(?<info_msg>[^\t]+\t(?<filename>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+))"
| search src_ip="192.168.202.79"
| stats count as events, dc(dest_ip) as targets, dc(uri) as unique_uris
```

```

<info_msg>[^\t]+\t(?<filename>
>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+)"
| search src_ip="192.168.202.79" (uri="*./*" OR uri="*etc/passwd*" OR uri="*boot.ini*" OR uri="*vmInventory*"
OR uri="*WPnBr.dll*" OR
uri="*<script>*" ORreferrer="*<script>*" ORreferrer="*system(*)"
| stats count

```

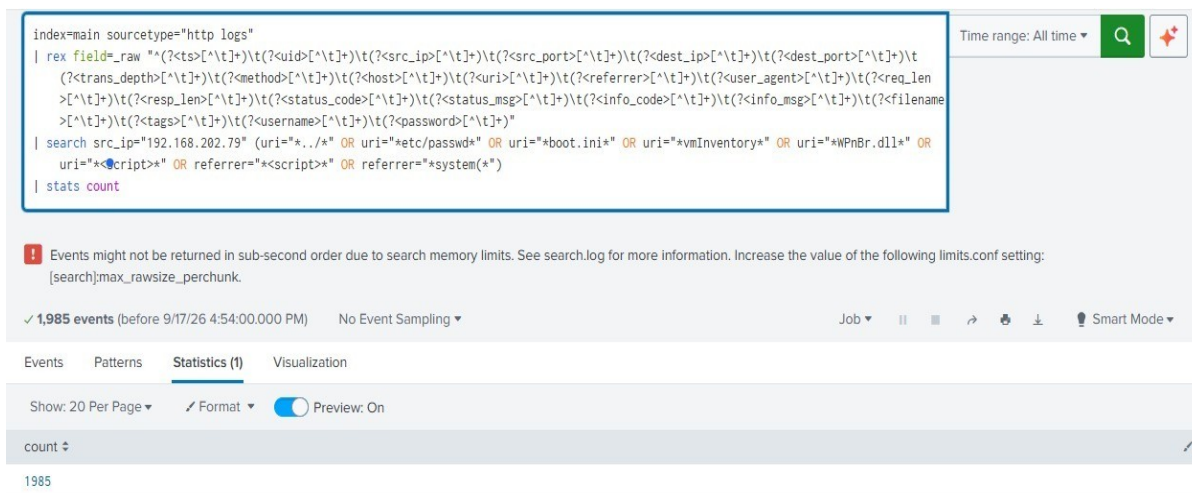


Figure 6 — 1,985 of this source's requests carried real attack payloads — directory traversal (./), attempts to read /etc/passwd and boot.ini, a VMware config exfiltration path, a known IIS exploit path (WPnBr.dll), and XSS/command-injection strings embedded directly in the URL.

Investigation approach: After finding multiple exploitation patterns in the HTTP requests, I expanded the investigation to Zeek's HTTP attack tags. I specifically checked for SQL-injection-tagged events to determine whether another source was performing SQL injection during the same incident window.

4.3.1 SQL Injection Attempts (192.168.202.110)

SQL injection is an attack where malicious database commands are inserted into a normal-looking web request (a URL or form field), tricking the website's database into running them. If successful, it can let an attacker read, modify, or delete data they should never have access to — or in some cases bypass a login entirely.

```

index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t
(?<trans_depth>[^\t]+\t(?<method>[^\t]+\t(?<host>[^\t]+\t(?<uri>[^\t]+\t(?<referrer>[^\t]+\t(?<user_agent>[^\t]
+)\t(?<req_len>
>[^\t]+\t(?<resp_len>[^\t]+\t(?<status_code>[^\t]+\t(?<status_msg>[^\t]+\t(?<info_code>[^\t]+\t(?<info_msg>[^\t]+\t(?<filename>
>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+)"
| search tags="HTTP::URI_SQLI"
| stats count by src_ip
| sort -count

```

```
index="main" sourcetype="http logs"
| rex field=_raw "^(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t
(?<trans_depth>[^\t]+\t(?<method>[^\t]+\t(?<host>[^\t]+\t(?<uri>[^\t]+\t(?<referrer>[^\t]+\t(?<user_agent>[^\t]+\t(?<req_len
>[^\t]+\t(?<resp_len>[^\t]+\t(?<status_code>[^\t]+\t(?<status_msg>[^\t]+\t(?<info_code>[^\t]+\t(?<info_msg>[^\t]+\t(?<filename
>[^\t]+\t(?<tags>[^\t]+\t(?<username>[^\t]+\t(?<password>[^\t]+)"
| search tags="HTTP::URI_SQLI"
| stats count by src_ip
| sort -count|
```

! Events might not be returned in sub-second order due to search memory limits. See search.log for more information. Increase the value of the following limits.conf setting:
[search]max_rawsize_perchunk.

✓ **2,827 events** (before 9/17/26 4:54:49.000 PM) No Event Sampling ▼
Job ▼ || ▢ → 🖨 ⬇️ 🧠 Smart Mode ▼

Events Patterns **Statistics (9)** Visualization

Show: 20 Per Page ▼
Format ▼ ☒ Preview: On

src_ip ↕	count ↕
192.168.202.110	2217
192.168.202.138	498
192.168.202.125	66
192.168.202.102	25
192.168.202.79	9
192.168.202.118	1

Investigation approach: A Zeek SQL-injection tag is an indicator, but I wanted to validate it using the actual HTTP request. I therefore inspected the raw URI values from the identified source to confirm whether real SQL-injection syntax was present rather than relying only on the tag.

4.3.2 Proof — Raw SQL Injection Payloads (192.168.202.110)

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t+)]\t(?<uid>[^\t+)]\t(?<src_ip>[^\t+)]\t(?<src_port>[^\t+)]\t(?<dest_ip>[^\t+)]\t(?<dest_port>[^\t+)]\t(?<trans_depth>[^\t+)]\t(?<method>[^\t+)]\t(?<host>[^\t+)]\t(?<uri>[^\t+)]\t(?<referrer>[^\t+)]\t(?<user_agent>[^\t+)]\t(?<req_len>[^\t+)]\t(?<resp_len>[^\t+)]\t(?<status_code>[^\t+)]\t(?<status_msg>[^\t+)]\t(?<info_code>[^\t+)]\t(?<info_msg>[^\t+)]\t(?<filename>[^\t+)]\t(?<tags>[^\t+)]\t(?<username>[^\t+)]\t(?<password>[^\t+)])"
| search tags="HTTP::URI_SQLI" src_ip="192.168.202.110"
| table _time, uri
| head 20
```

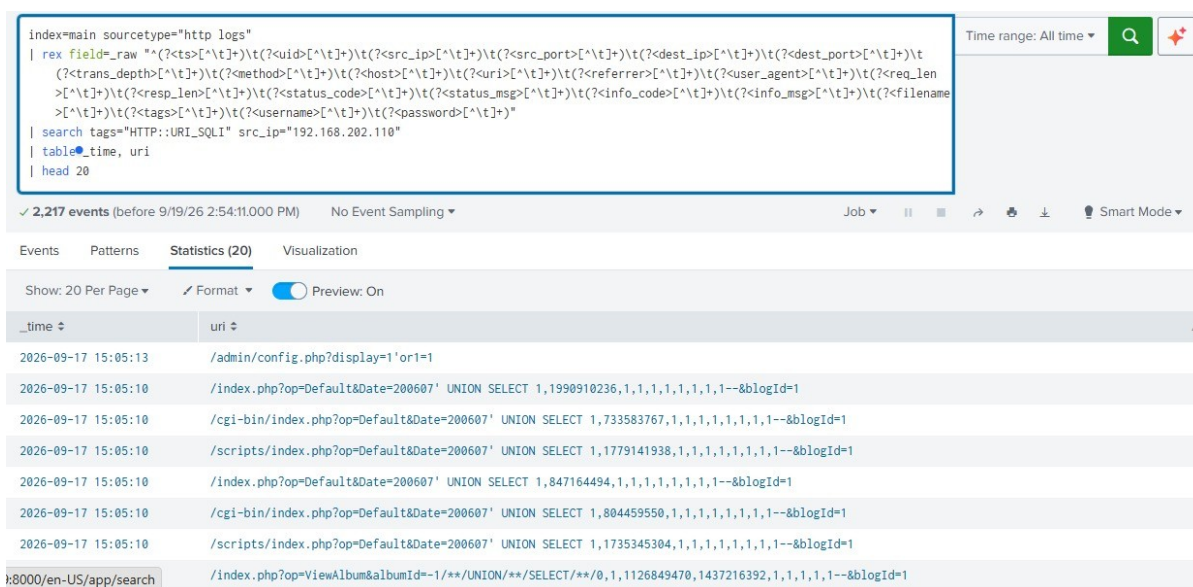


Figure 8 — Actual request URIs from 192.168.202.110, showing real SQL injection syntax embedded directly in the URL — UNION SELECT statements and a classic 1'or'1=1 tautology — not just a tag, but the literal attack payload itself.

Investigation approach: After confirming the actual SQL-injection payloads, I checked the HTTP response status codes to understand how the targeted servers responded to those requests. This distinguishes attempted activity from evidence about how the servers responded.

4.3.3 Outcome of the SQL Injection Attempts (192.168.202.110)

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t+])\t(?<uid>[^\t+])\t(?<src_ip>[^\t+])\t(?<src_port>[^\t+])\t(?<dest_ip>[^\t+])\t(?<dest_port>[^\t+])\t(?<trans_depth>[^\t+])\t(?<method>[^\t+])\t(?<host>[^\t+])\t(?<uri>[^\t+])\t(?<referrer>[^\t+])\t(?<user_agent>[^\t+])\t(?<req_len>[^\t+])\t(?<resp_len>[^\t+])\t(?<status_code>[^\t+])\t(?<status_msg>[^\t+])\t(?<info_code>[^\t+])\t(?<info_msg>[^\t+])\t(?<filename>[^\t+])\t(?<tags>[^\t+])\t(?<username>[^\t+])\t(?<password>[^\t+])"
| search tags="HTTP::URI_SQLI" src_ip="192.168.202.110"
| stats count by status_code
```

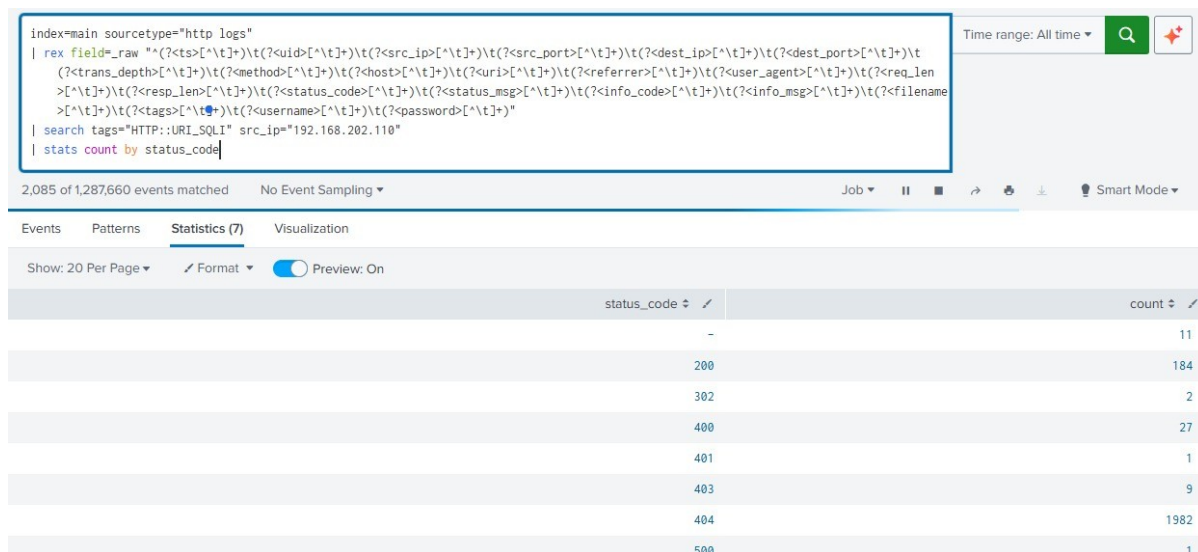


Figure 9 — Of this source's 2,085 SQL injection attempts, 1,982 returned 404 and were blocked, but 184 returned 200 OK — meaning the server processed and responded normally to those requests, and 9 returned 403 Forbidden, suggesting some form of access-control response.

Investigation approach: Since the investigation had already identified several exploitation techniques, I also checked for credential-guessing activity against known web application paths. I searched for repeated use of the guest username and examined the destination hosts and URI involved.

4.3.4 Credential Brute-Force Against Nagios (192.168.202.96)

```
index=main sourcetype="http logs"
| rex field=_raw "(?<ts>[^\t+])\t(?<uid>[^\t+])\t(?<src_ip>[^\t+])\t(?<src_port>[^\t+])\t(?<dest_ip>[^\t+])\t(?<dest_port>[^\t+])\t(?<trans_depth>[^\t+])\t(?<method>[^\t+])\t(?<host>[^\t+])\t(?<uri>[^\t+])\t(?<referrer>[^\t+])\t(?<user_agent>[^\t+])\t(?<req_len>[^\t+])\t(?<resp_len>[^\t+])\t(?<status_code>[^\t+])\t(?<status_msg>[^\t+])\t(?<info_code>[^\t+])\t(?<info_msg>[^\t+])\t(?<filename>[^\t+])\t(?<tags>[^\t+])\t(?<username>[^\t+])\t(?<password>[^\t+])"
```

```
| search src_ip="192.168.202.96" username="guest"
| stats count by uri, dest_ip
```

The screenshot shows the Splunk search interface. At the top, a search bar contains the query: `index=main sourcetype="http logs"`. Below the search bar, a list of fields is displayed, including `field=raw`, `field=source`, `field=host`, `field=uri`, `field=referrer`, `field=user_agent`, `field=req_len`, `field=resp_len`, `field=status_code`, `field=status_msg`, `field=info_code`, `field=info_msg`, and `field=filename`. The search results are displayed in a table with columns: `uri`, `dest_ip`, and `count`. The table shows 8 events, all with a count of 1. The events are:

uri	dest_ip	count
/nagios3/cgi-bin/statuswml.cgi	192.168.22.152	1
/nagios3/cgi-bin/statuswml.cgi	192.168.24.103	1
/nagios3/cgi-bin/statuswml.cgi	192.168.24.152	1
/nagios3/cgi-bin/statuswml.cgi	192.168.25.152	1

Figure 8 — 8 attempts using username “guest” were made from 192.168.202.96, all against /nagios3/cgi-bin/statuswml.cgi — a known Nagios exploit path — across 5 different destination hosts.

Taken together, the enumeration sweep and injection payloads from .79, the SQL injection attempts concentrated on .110, and the Nagios credential guessing from .96 show multiple IPs in the same incident window moving beyond reconnaissance into active exploitation attempts — reading sensitive files, injecting code and database commands, and guessing admin credentials.

5. Overall Finalization

Each attack was validated using an evidence checklist rather than a single indicator. The table below summarizes which criteria were confirmed, based only on the queries and screenshots gathered above.

Attack	Criteria	Evidence
Attack #1	High-volume automated scanning	1,284,535 requests from a single source — over 62% of the entire log
Attack #1	Tool signature confirmed	DirBuster-0.12 on 1,284,281 of 1,284,535 requests
Attack #1	Concentrated targeting	1,284,397 requests aimed at one host, 192.168.229.101
Attack #1	Blind enumeration confirmed	1,284,319 of 1,284,535 requests (99.98%) returned 404
Attack #2	Web enumeration confirmed	232,259 requests, 56 targets, 210,554 unique URIs from 192.168.202.79
Attack #2	Injection / traversal payloads	1,985 requests carrying real attack payloads (traversal, XSS, command injection)
Attack #2	SQL injection confirmed	2,827 Zeek-tagged HTTP::URI_SQLI events, 2,217 from 192.168.202.110
Attack #2	SQL injection payload confirmed	Raw URIs show real UNION SELECT and 1'or1=1 syntax, not just a tag
Attack #2	SQL injection outcome	Of 2,085 attempts from .110: 1,982 blocked (404), 184 returned 200 OK
Attack #2	Credential brute-force confirmed	8 “guest” login attempts from 192.168.202.96 against a Nagios exploit path

Final Verdict: Both attacks are confirmed by the evidence gathered. Attack #1 (192.168.203.63) is a clear, isolated, tool-signed directory brute-force scan — the single largest source of traffic in the entire log by a wide margin. Attack #2 is a separate, multi-IP campaign centered on 192.168.202.79 that moved beyond scanning into active exploitation: real injection and traversal payloads, a distinct SQL injection campaign from 192.168.202.110 with actual UNION SELECT payloads and 184 requests returning 200 OK, and credential guessing against a known Nagios exploit path from 192.168.202.96. Together these represent two distinct, confirmed attack patterns within the same incident window.